



数据结构

Data Structure

许 蕾

xlei@nju.edu.cn

南京大学计算机学院



第九章 排序



- 概述
- 插入排序
- 快速排序
- 交换排序（起泡排序）
- 选择排序
- 归并排序



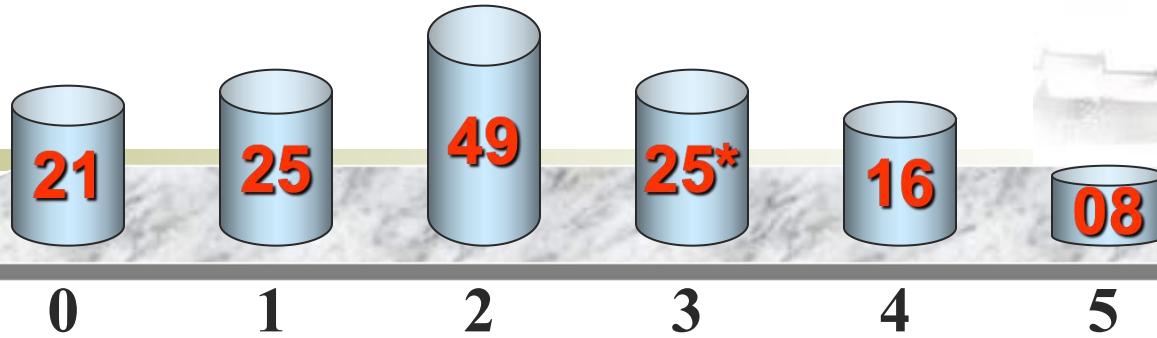
交换排序 (Exchange Sort)



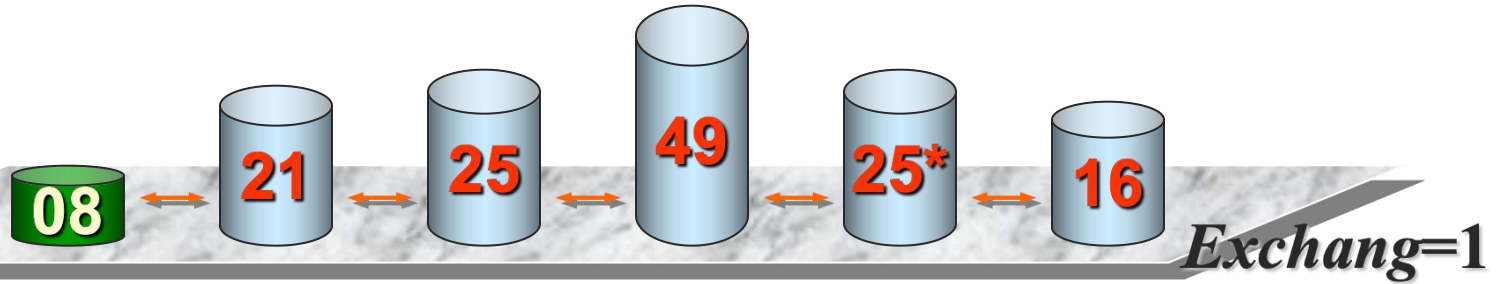
- 基本思想是两两比较待排序元素的排序码，如果发生逆序(即排列顺序与排序后的次序正好相反)，则交换之。直到所有元素都排好序为止。

起泡排序 (Bubble Sort)

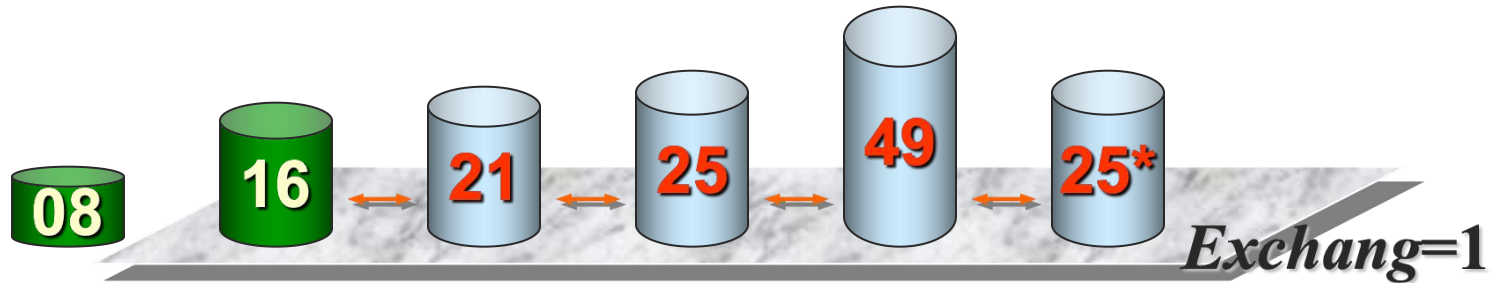
- 基本方法是：设待排序元素序列中的元素个数为 n 。最多作 $n-1$ 趟， $i = 1, 2, \dots, n-1$ 。在第 i 趟中从后向前， $j = n-1, n-2, \dots, i$ ，顺次两两比较 $V[j-1].key$ 和 $V[j].key$ 。如果发生逆序，则交换 $V[j-1]$ 和 $V[j]$ 。
- 又叫冒泡排序。



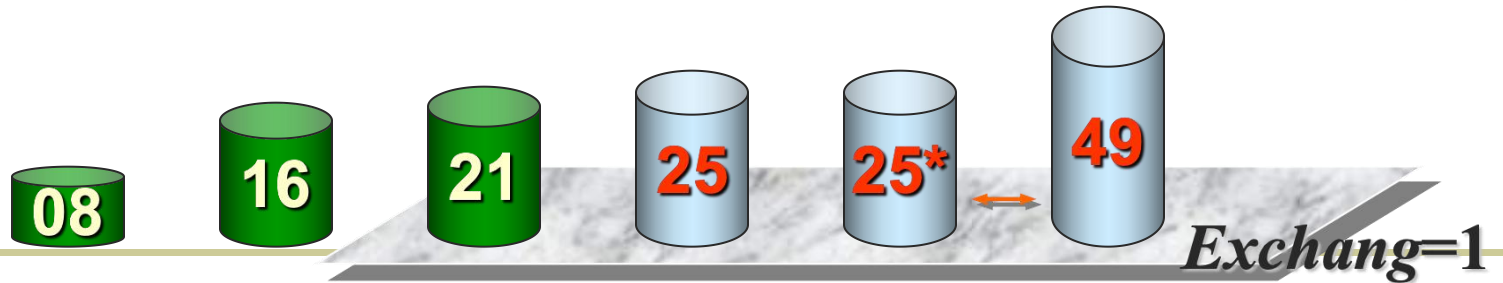
$i = 1$



$i = 2$

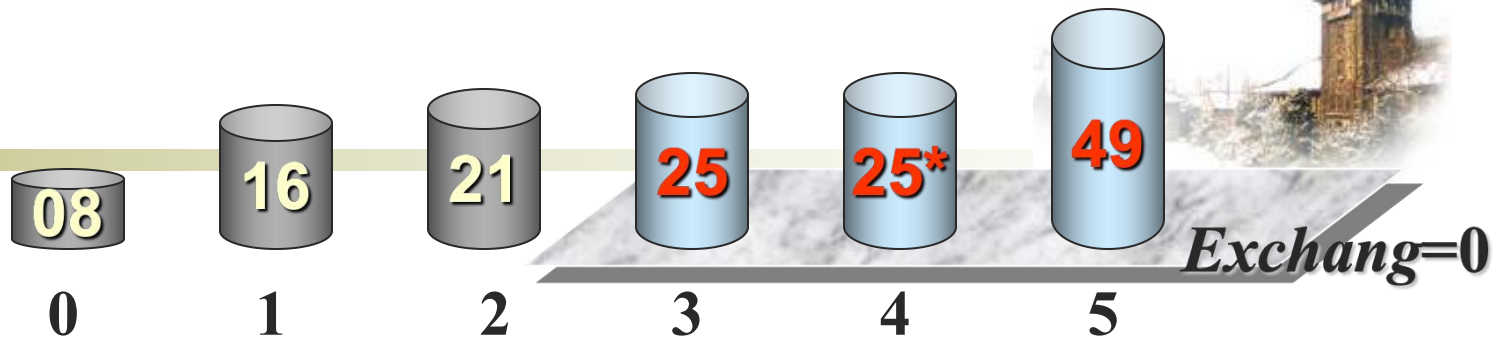


$i = 3$





$l = 4$



起泡排序的算法

```
template <class T>
```

```
void BubbleSort (dataList<T>& L, const int left,  
                 const int right) {
```

```
    int pass = left+1, exchange = 1;
```

```
    while (pass <= right && exchange) {
```

```
        exchange = 0;                                //标志为0假定未交换
```

```
        for (int j = right; j >= pass; j--)
```



```
if (L[j-1] > L[j]) { //逆序
```

```
    Swap (L[j-1], L[j]);    //交换
```

```
    exchange = 1;          //标志置为1,有交换
```

```
}
```

```
pass++;
```

```
}
```

```
};
```

算法分析

- 第 i 趟($1 \leq i < n$)对待排序元素序列 $V[i-1], V[i], \dots, V[\text{right}]$ 进行排序, 结果将该序列中排序码最小的元素交换到序列的第一个位置($i-1$)。



最多做 $n-1$ 趟起泡就能把所有元素排好序。

- **最好的情形**是在元素的初始排列已经按排序码从小到大排好序时，此算法**只执行一趟起泡**，做 $n-1$ 次排序码比较，不移动元素。
- **最坏的情形**是算法执行 $n-1$ 趟起泡，第 i 趟 ($1 \leq i < n$) 做 $n-i$ 次排序码比较，执行 $n-i$ 次元素交换。在最坏情形下总的排序码比较次数 KCN 和元素移动次数 RMN 为：

$$KCN = \sum_{i=1}^{n-1} (n-i) = \frac{1}{2}n(n-1)$$

$$RMN = 3 \sum_{i=1}^{n-1} (n-i) = \frac{3}{2}n(n-1)$$

- 起泡排序需要一个附加元素以实现元素值的对换。起泡排序是一个**稳定**的排序方法。



9.4 选择排序



- 基本思想是：每一趟（例如第 i 趟, $i = 0, 1, \dots, n-2$ ）在后面 $n-i$ 个待排序元素中选出排序码最小的元素，作为有序元素序列的第 i 个元素。待到第 $n-2$ 趟做完，待排序元素只剩下1个，就不用再选了。
 - 直接选择排序
 - 锦标赛排序
 - 堆排序



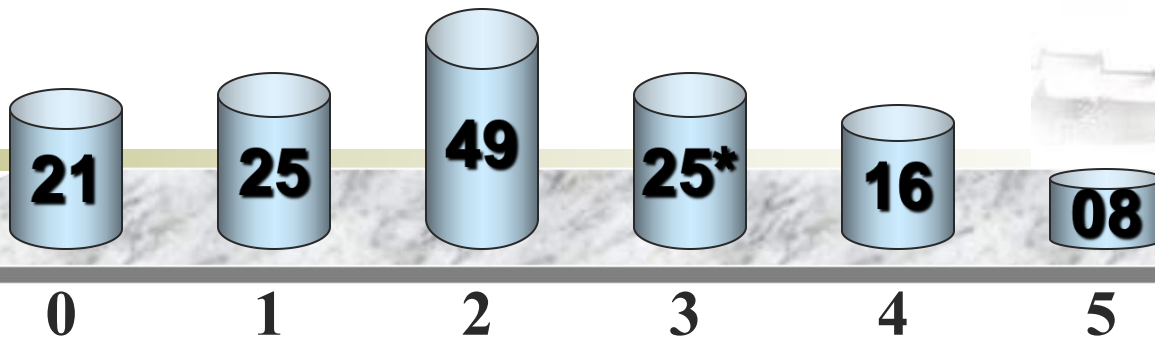
9.4.1 直接选择排序 (Select Sort)



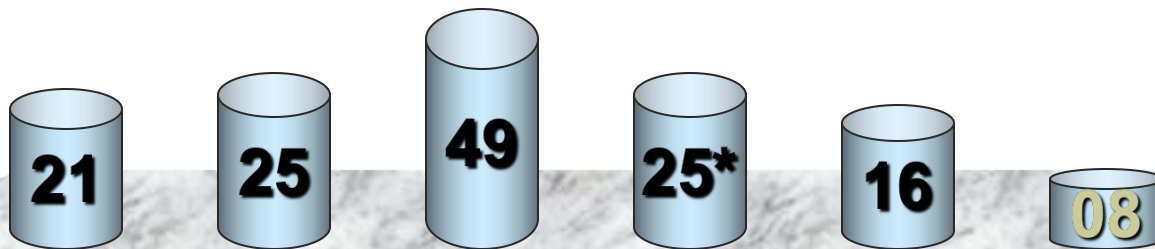
- 直接选择排序的基本步骤是：
 - ① 在一组元素 $V[i] \sim V[n-1]$ 中选择具有最小排序码的元素；
 - ② 若它不是这组元素中的第一个元素, 则将它与这组元素中的第一个元素对调；
 - ③ 在这组元素中剔除这个具有最小排序码的元素。在剩下的元素 $V[i+1] \sim V[n-1]$ 中重复执行第①、②步, 直到剩余元素只有一个为止。



初始

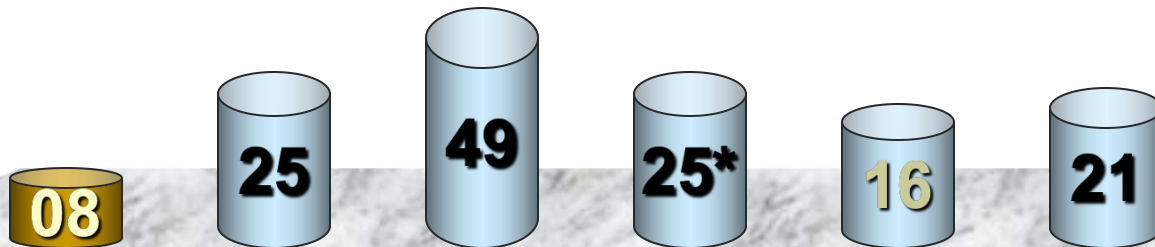


$i = 0$



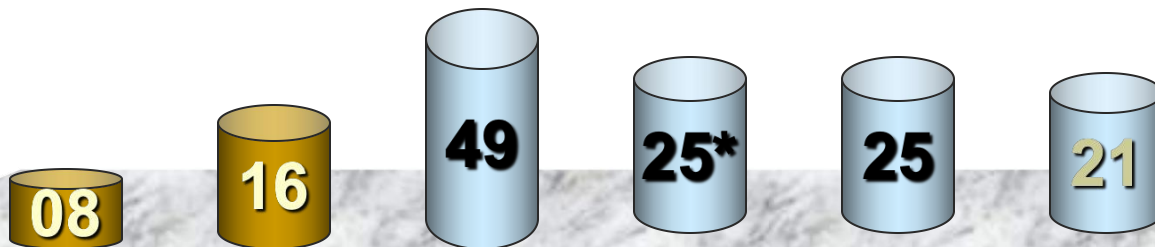
最小者 08
交换 21, 08

$i = 1$



最小者 16
交换 25, 16

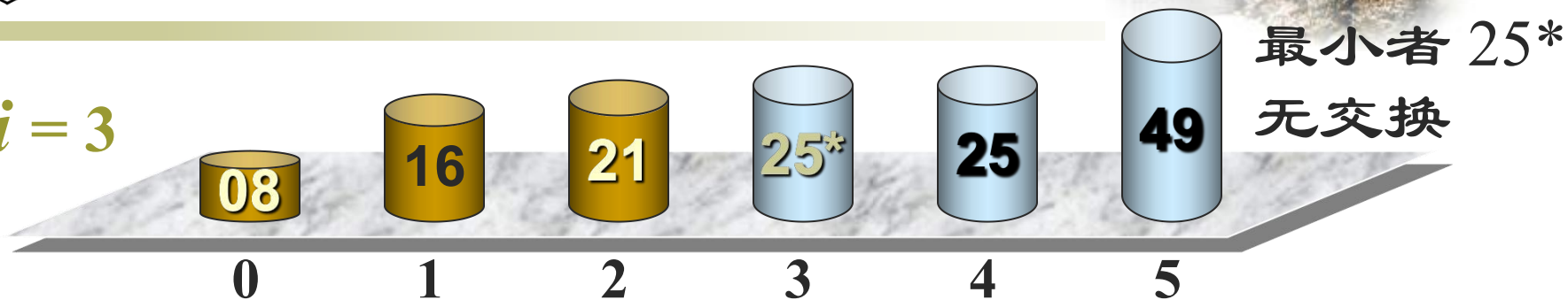
$i = 2$



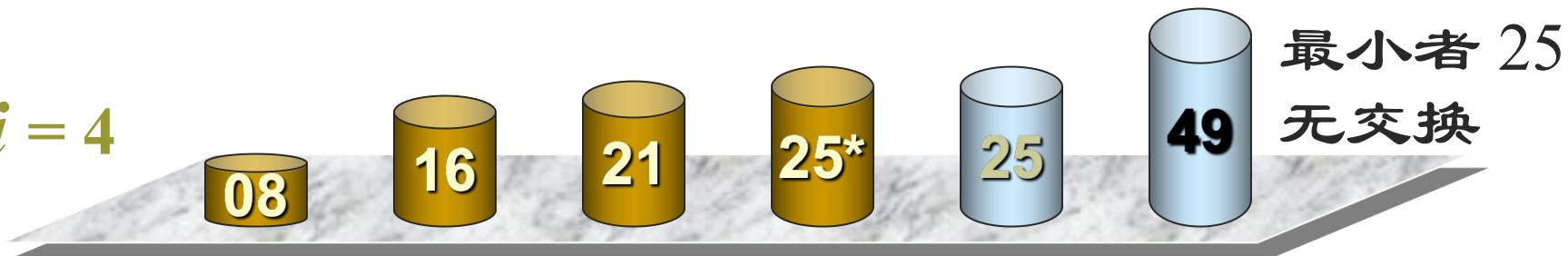
最小者 21
交换 49, 21



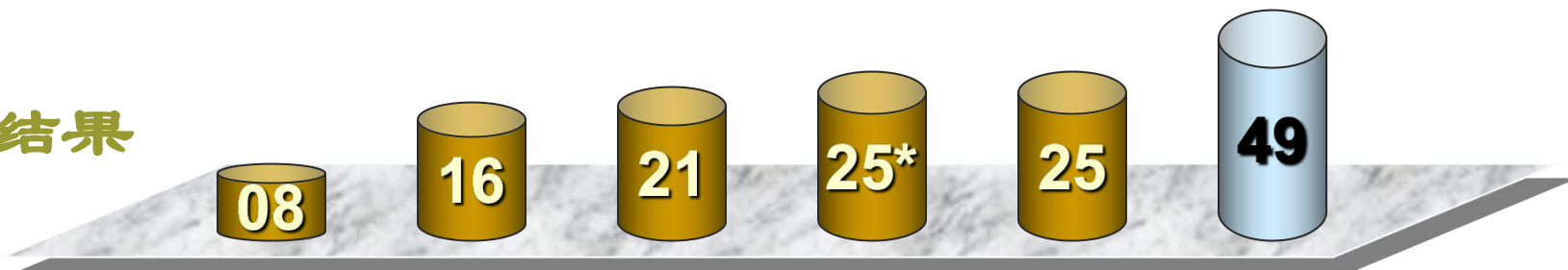
$i = 3$



$i = 4$



结果



各趟排序后的结果



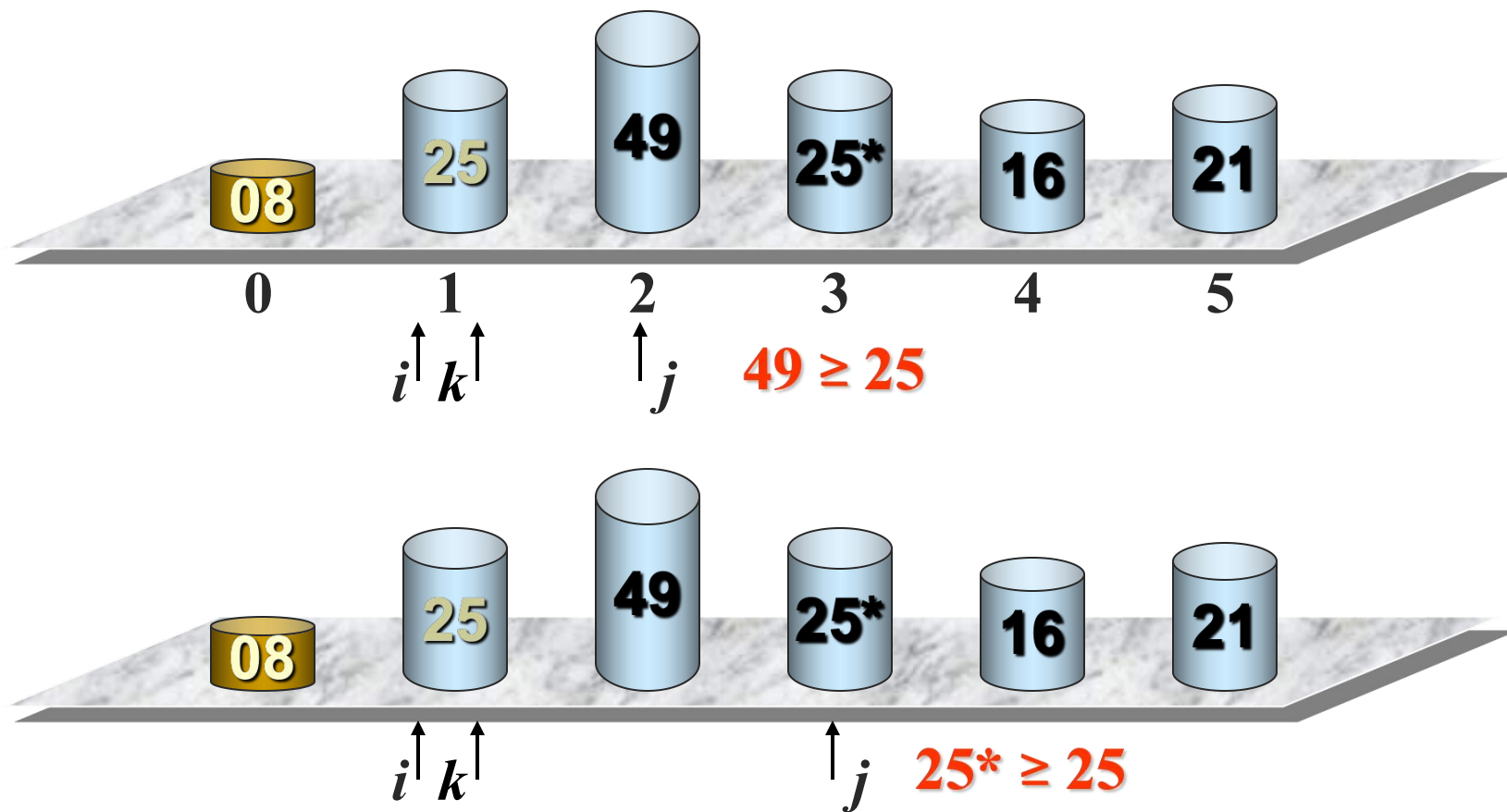
直接选择排序的算法

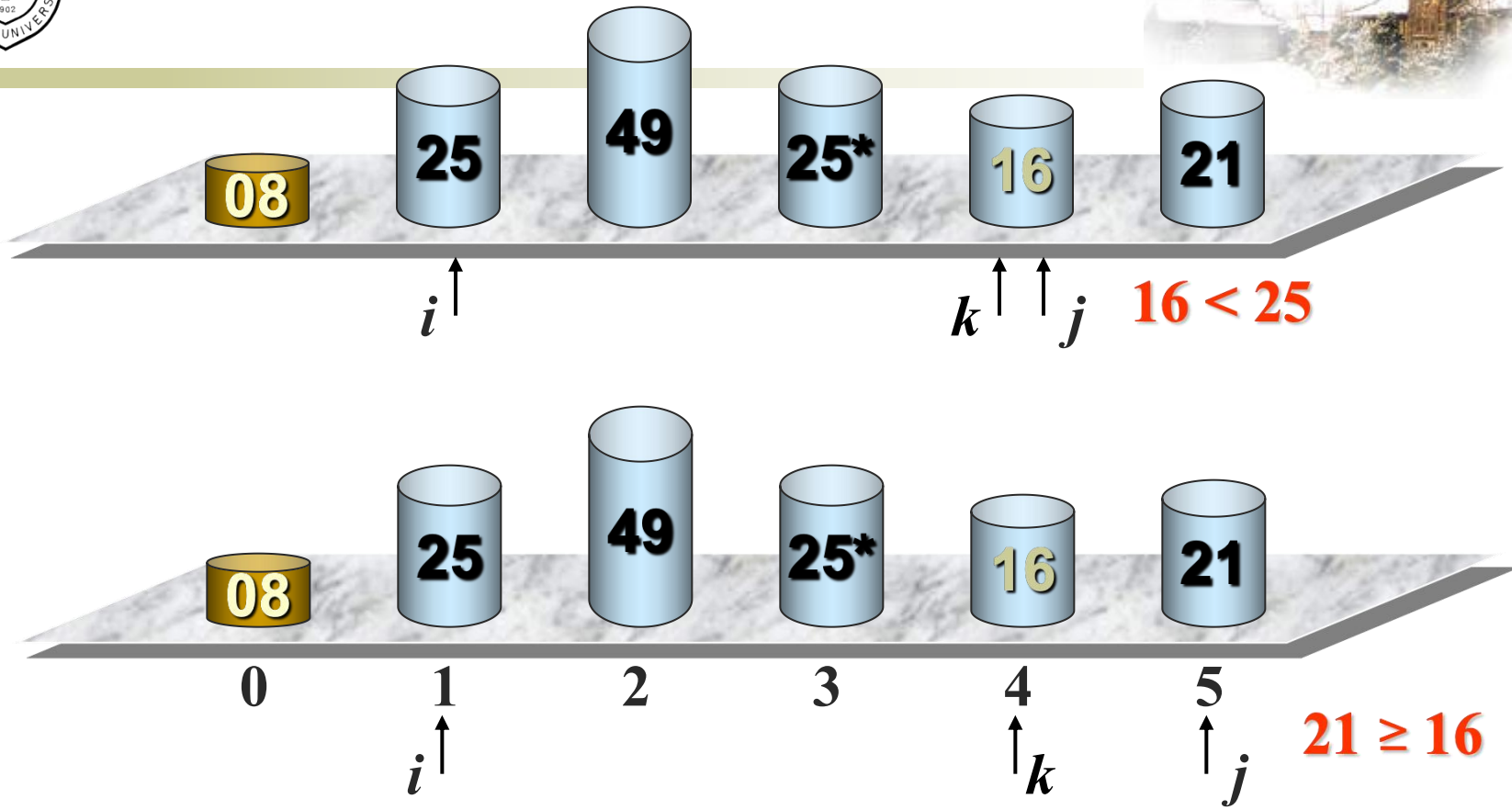


```
#include "dataList.h"
template <class T>
void SelectSort (dataList<T>& L,
    const int left, const int right) {
    for (int i = left; i < right; i++) {
        int k = i;    //在L[i]到L[right]之间找最小排序码元素
        for (int j = i+1; j <= right; j++)
            if (L[j] < L[k]) k = j;
        if (k != i) Swap (L[i], L[k]); //交换
    }
};
```



$i=1$ 时选择排序的过程





k 指示当前序列中最小者



- 直接选择排序的排序码比较次数 KCN 与元素的初始排列无关。设整个待排序元素序列有 n 个元素，则第 i 趟选择具有最小排序码元素所需的比较次数总是 $n-i-1$ 次。总的排序码比较次数为

$$KCN = \sum_{i=0}^{n-2} (n-i-1) = \frac{n(n-1)}{2}$$

- 元素移动次数与元素序列初始排列有关。
 - 最好情况是这组元素初始状态是按其排序码从小到大有序排列，元素的移动次数达到最少， $RMN=0$ ；
 - 最坏情况是每一趟都要进行交换，总的元素移动次数为 $RMN=3(n-1)$ 。
- 直接选择排序是一种不稳定的排序方法。



9.4.2 锦标赛排序 (Tournament Sort)

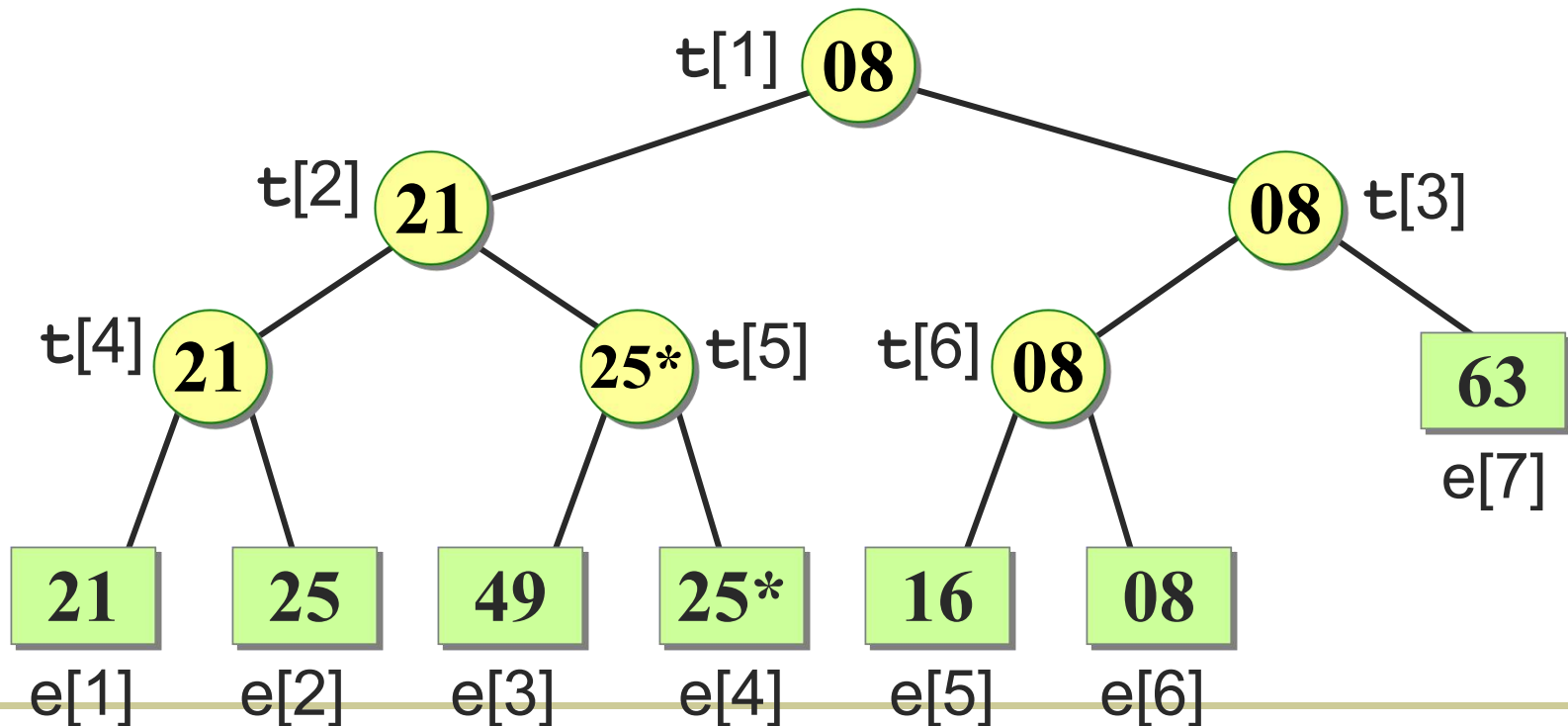


- 思想与体育比赛时的淘汰赛类似。
- 首先取得 n 个元素的排序码, 进行两两比较, 得到 $\lceil n/2 \rceil$ 个比较的优胜者(排序码小者), 作为第一步比较的结果保留下来。
- 然后对这 $\lceil n/2 \rceil$ 个元素再进行排序码的两两比较, ..., 如此重复, 直到选出一个排序码最小的元素为止。
- 由于每次两两比较的结果是把排序码小者作为优胜者上升到双亲结点, 所以称这种比赛树为胜者树。



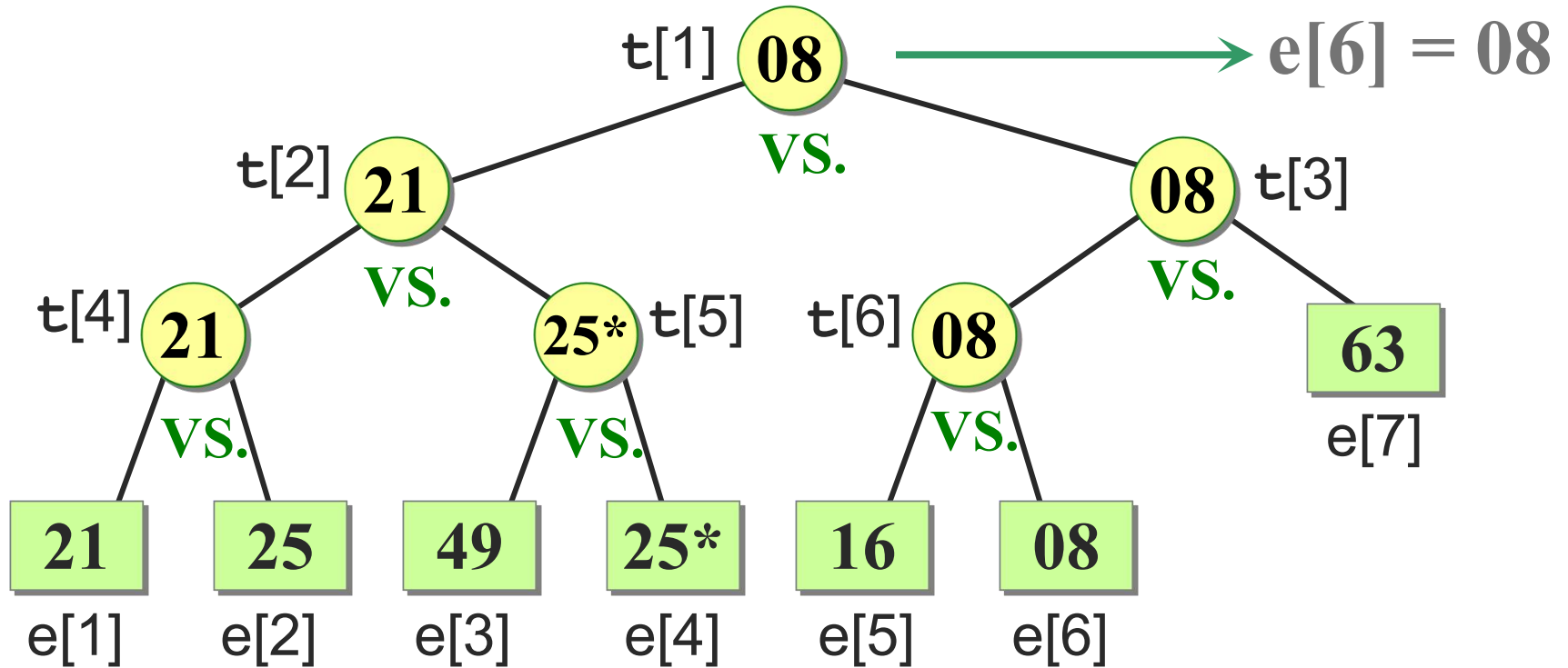
- 在图例中，最下面是元素排列的初始状态，相当于一棵完全二叉树的叶结点，它存放的是所有参加排序的元素的排序码。

Winner





Winner (胜者)

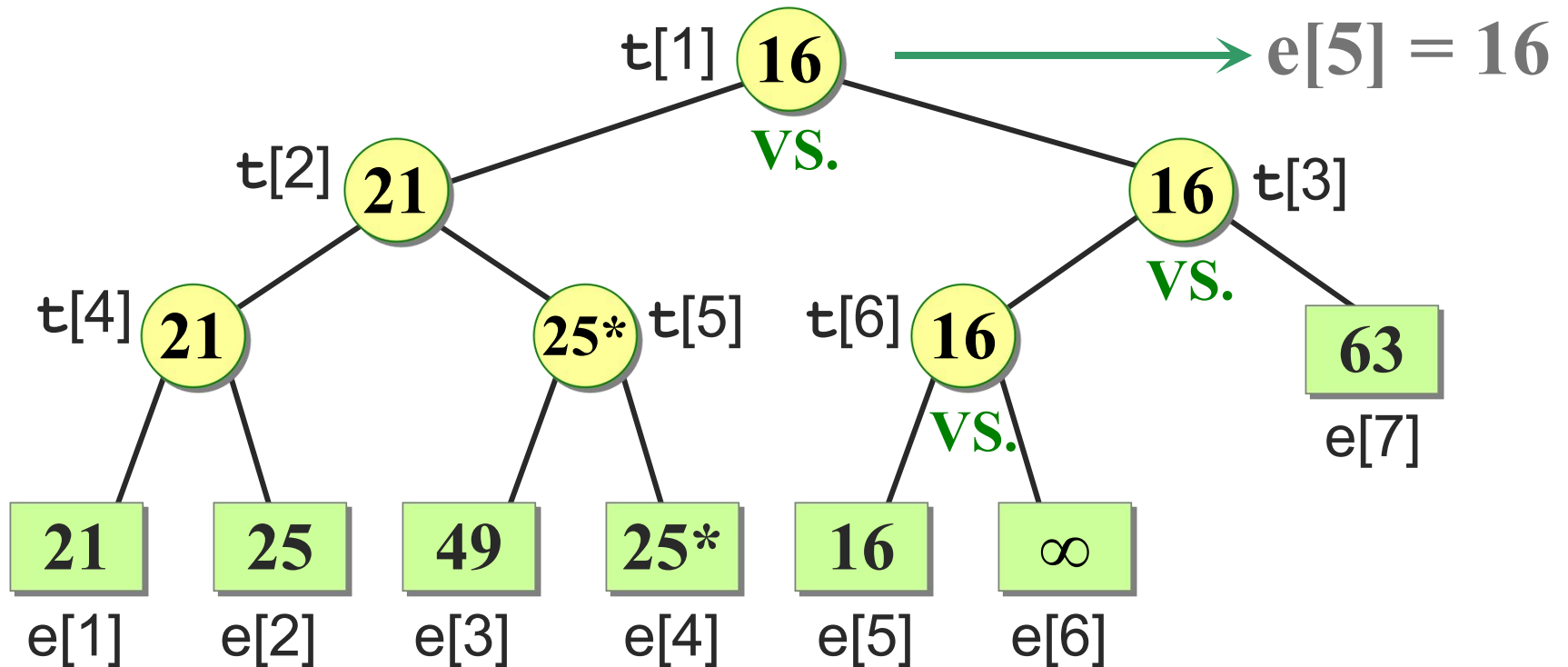


形成初始胜者树（最小排序码上升到根）

排序码比较次数：6



Winner (胜者)

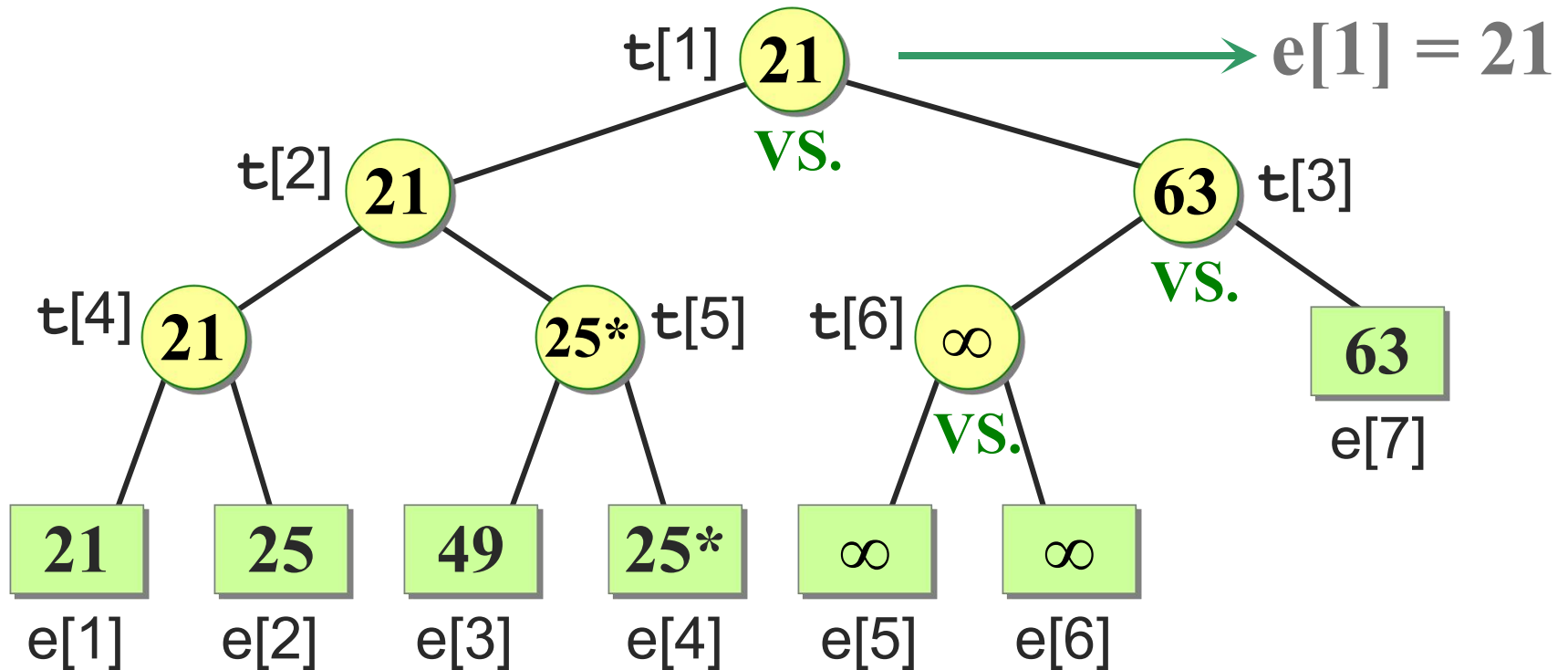


输出冠军 $e[6] = 08$ 并调整胜者树后树的状态

排序码比较次数：3



Winner (胜者)

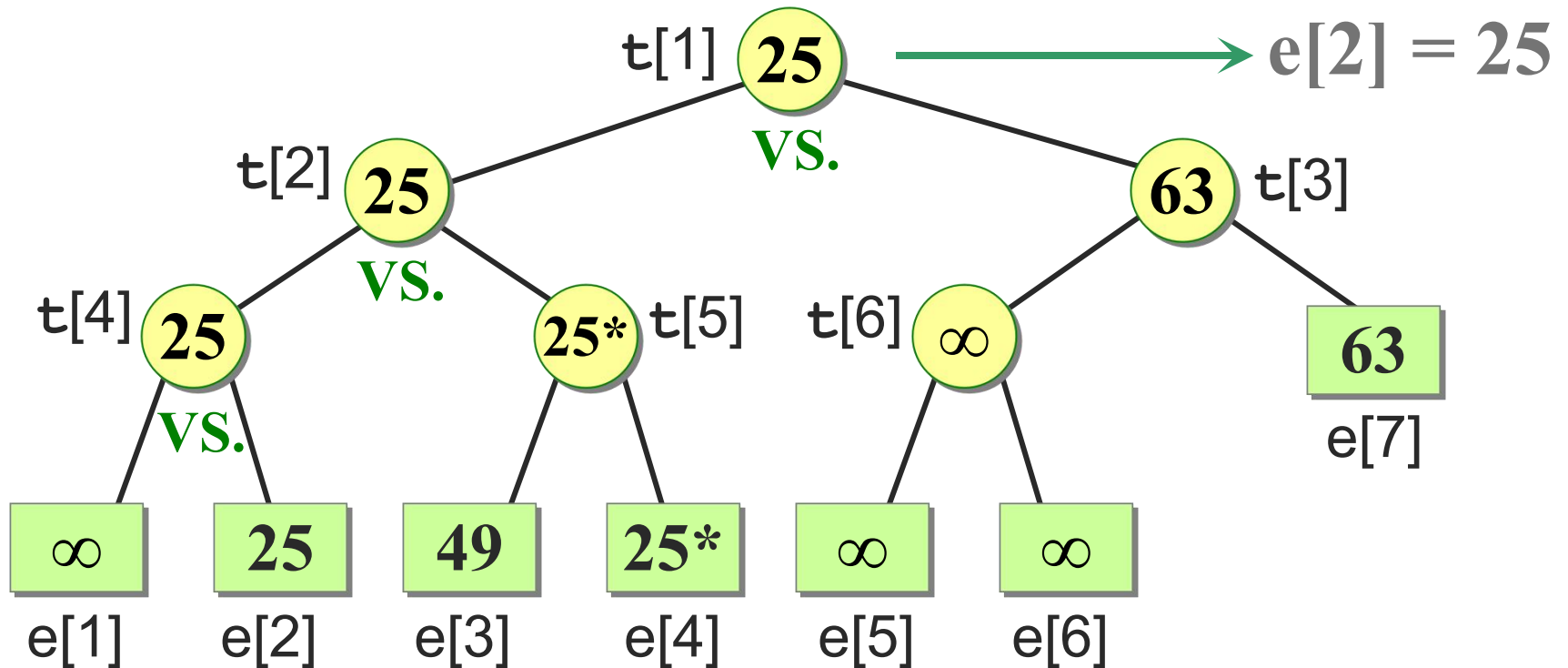


输出冠军 $e[5] = 16$ 并调整胜者树后树的状态

排序码比较次数：3



Winner (胜者)

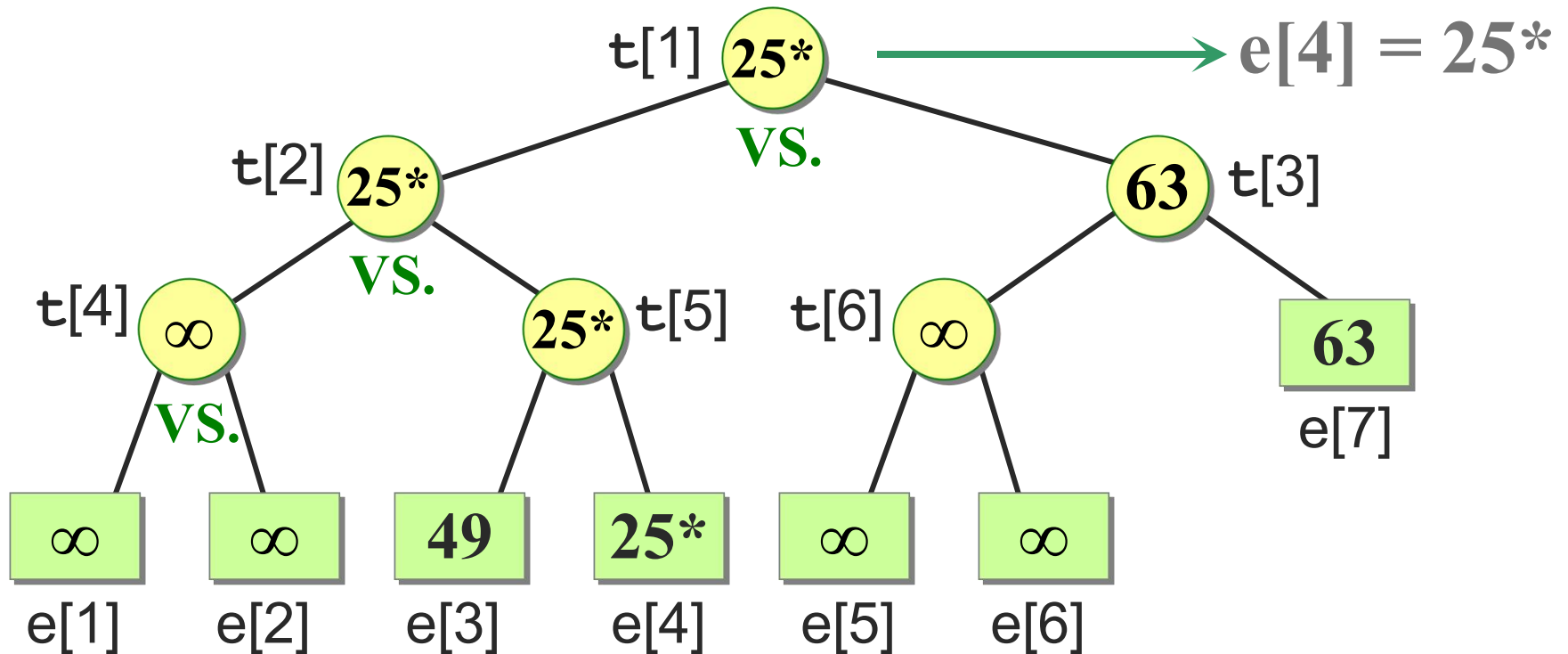


输出冠军 $e[1] = 21$ 并调整胜者树后树的状态

排序码比较次数：3



Winner (胜者)

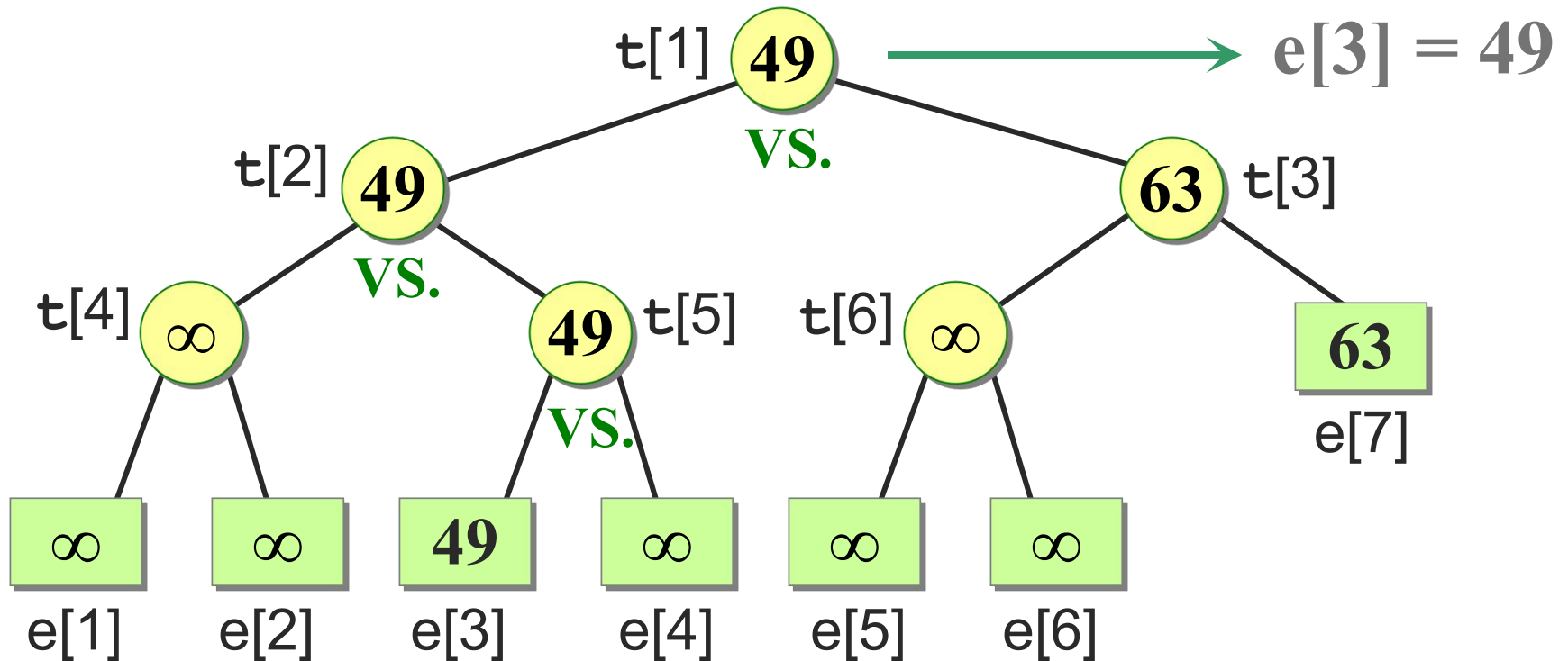


输出冠军e[2] = 25 并调整胜者树后树的状态

排序码比较次数：3



Winner (胜者)

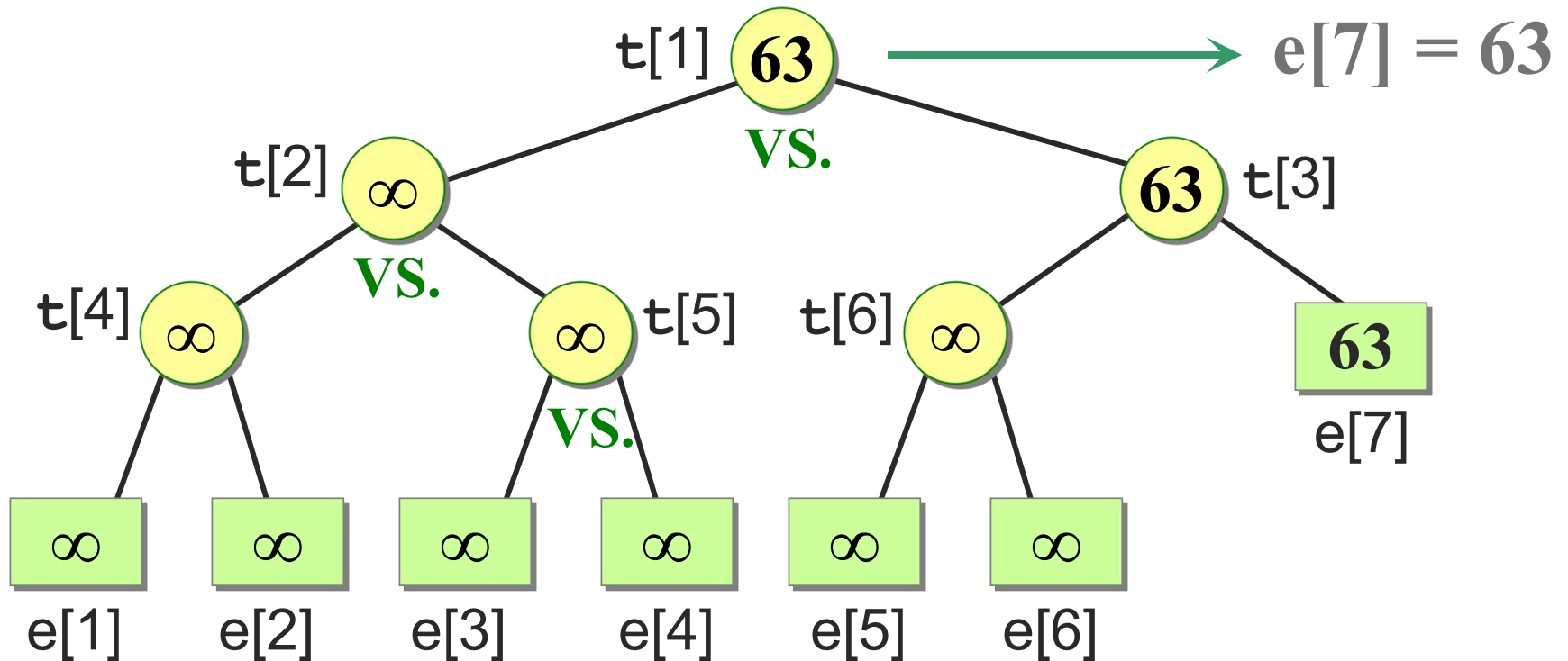


输出冠军 $e[4] = 25^*$ 并调整胜者树后树的状态

排序码比较次数：3



Winner (胜者)



输出冠军 $e[3] = 49$ 并调整胜者树后树的状态

排序码比较次数：3



- 胜者树是完全二叉树，其高度为 $\lceil \log_2 n \rceil$ ，其中 n 为待排序元素个数。
- 除第一次选择具有最小排序码的元素需要进行 $n-1$ 次排序码比较外，重构胜者树选择次小、再次小排序码所需的比较次数均为 $O(\log_2 n)$ 。总排序码比较次数为 $O(n \log_2 n)$ 。



- 这种排序方法虽然减少了许多排序时间，但是使用了较多的附加存储。
- 如果有 n 个元素，必须使用至少 $2n-1$ 个结点来存放胜者树。
- 锦标赛排序是一个稳定的排序方法。



- 假设一个盲人程序员只能通过"比较器"（告诉你两个元素谁大谁小）来操作数据，不能直接看到数组。
- 请用最少的比较次数找出数组中的最大值和最小值；
- 设计一个"盲人友好"的排序算法（描述你的思考过程）



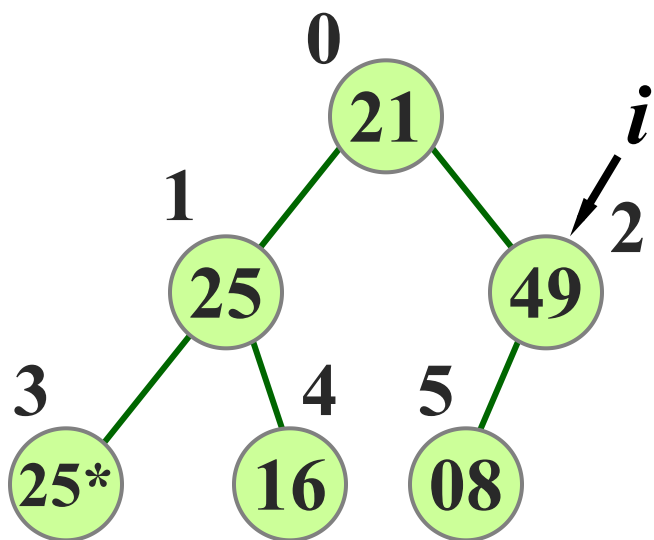
9.4.3 堆排序 (Heap Sort)



- 利用堆及其运算, 可以很容易地实现选择排序的思路。
- 堆排序分为两个步骤
 - ◆ 根据初始输入数据, 利用堆的调整算法 `siftDown()` 形成初始堆;
 - ◆ 通过一系列的元素交换和重新调整堆进行排序。
- 为了实现元素按排序码从小到大排序, 要求建立最大堆。

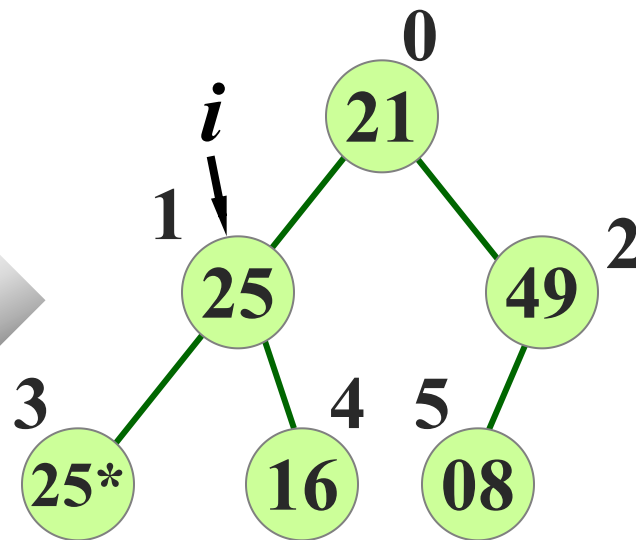


建立初始的最大堆



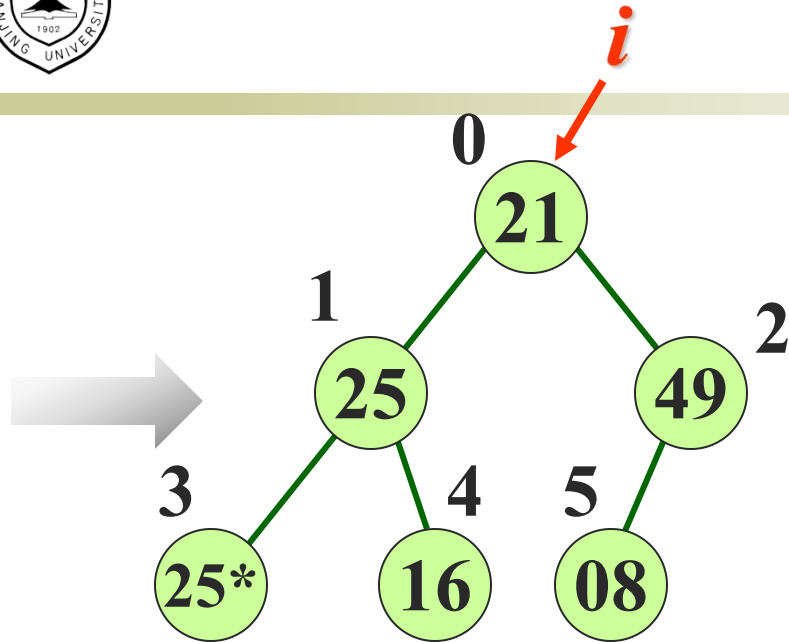
21	25	49	25*	16	08
----	----	----	-----	----	----

初始排序码集合



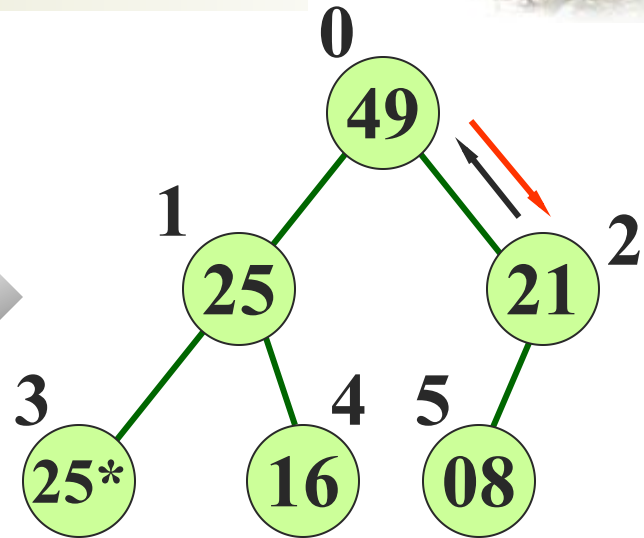
21	25	49	25*	16	08
----	----	----	-----	----	----

$i = 2$ 时的局部调整



21	25	49	25*	16	08
----	----	----	-----	----	----

$i = 1$ 时的局部调整



49	25	21	25*	16	08
----	----	----	-----	----	----

$i = 0$ 时的局部调整
形成最大堆



最大堆的向下调整算法



```
template <class T>
```

```
siftDown (dataList<T>& L, const int start, const int m){
```

```
//私有函数: 从结点start开始到m自上向下比较,
```

```
//如果子女的值大于双亲的值, 则相互交换, 将一
```

```
//个集合局部调整为最大堆。
```

```
    int i = start; int j = 2*i+1;
```

```
//j是i的左子女
```

```
    Element<T> temp = L[i];
```

```
//暂存子树根结点
```

```
    while (j <= m) {
```

```
//逐层比较
```

```
        if (j < m && L[j] < L[j+1]) j++;
```

```
//让j指向两子女中的大者
```

```
        if (temp >= L[j]) break;
```

```
//temp排序码大,不调整
```



```
else {  
    L[i] = L[j];  
    i = j; j = 2*j+1;  
}  
}  
L[i] = temp;  
};
```

//否则子女中的大者上移

//i下降到子女位置

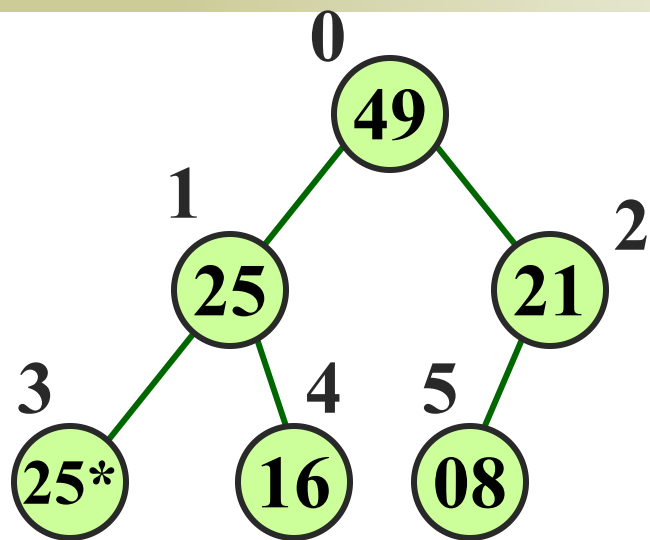
//temp放到合适位置



基于初始堆进行堆排序

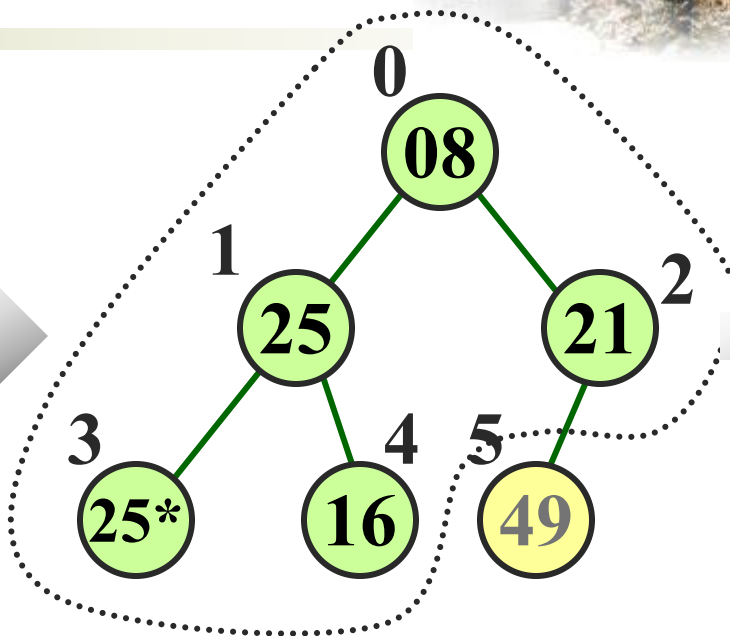


- 最大堆堆顶 $L.Vector[0]$ 具有最大的排序码，将 $L.Vector[0]$ 与 $L.Vector[n-1]$ 对调，把具有最大排序码的元素交换到最后，再对前面的 $n-1$ 个元素，使用堆的调整算法 $siftDown(L, 0, n-2)$ ，重新建立最大堆，具有次最大排序码的元素又上浮到 $L.Vector[0]$ 位置。
- 再对调 $L.Vector[0]$ 和 $L.Vector[n-2]$ ，再调用 $siftDown(L, 0, n-3)$ ，对前面的 $n-2$ 个元素重新调整，...
- 如此反复执行，最后得到全部排序好的元素序列。这个算法即堆排序算法，其细节在下面的程序中给出。



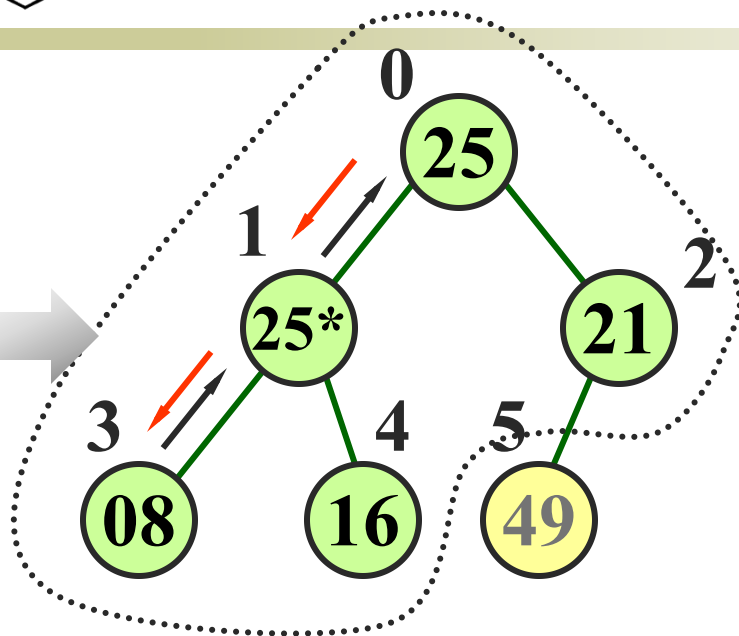
49	25	21	25*	16	08
----	----	----	-----	----	----

初始最大堆



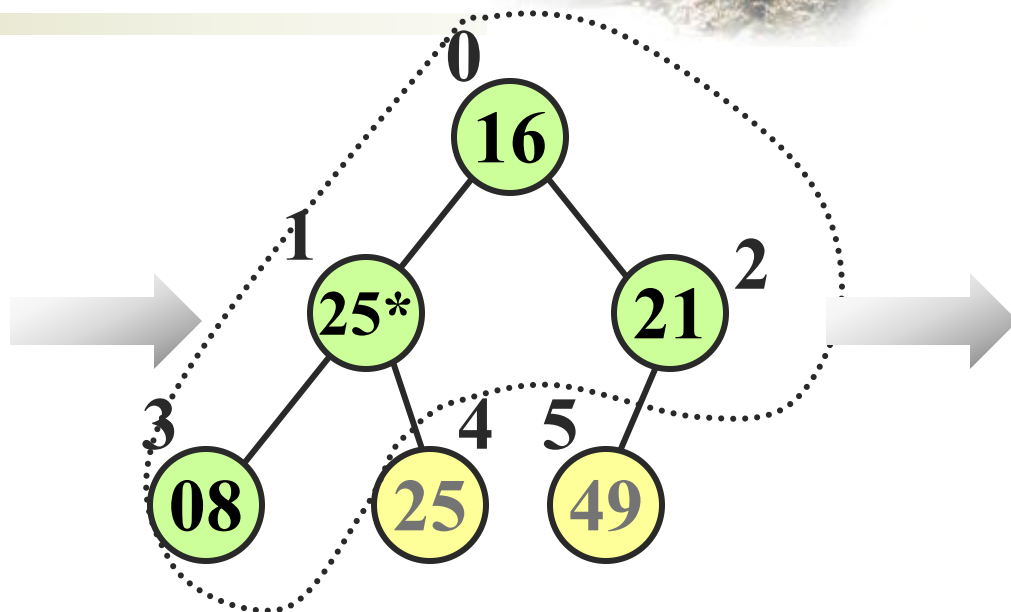
08	25	21	25*	16	49
----	----	----	-----	----	----

交换 0 号与 5 号元素，
5 号元素就位



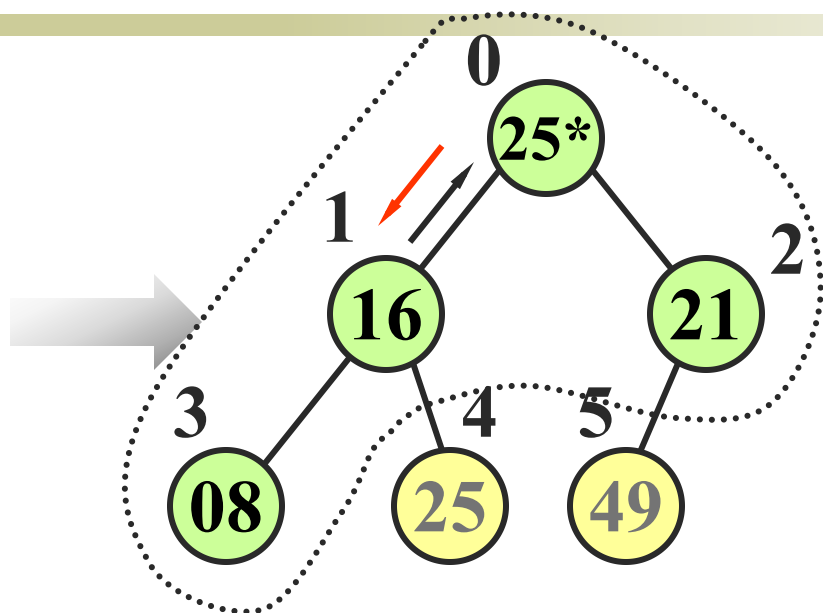
25	25*	21	08	16	49
----	-----	----	----	----	----

从 0 号到 4 号 重新
调整为最大堆



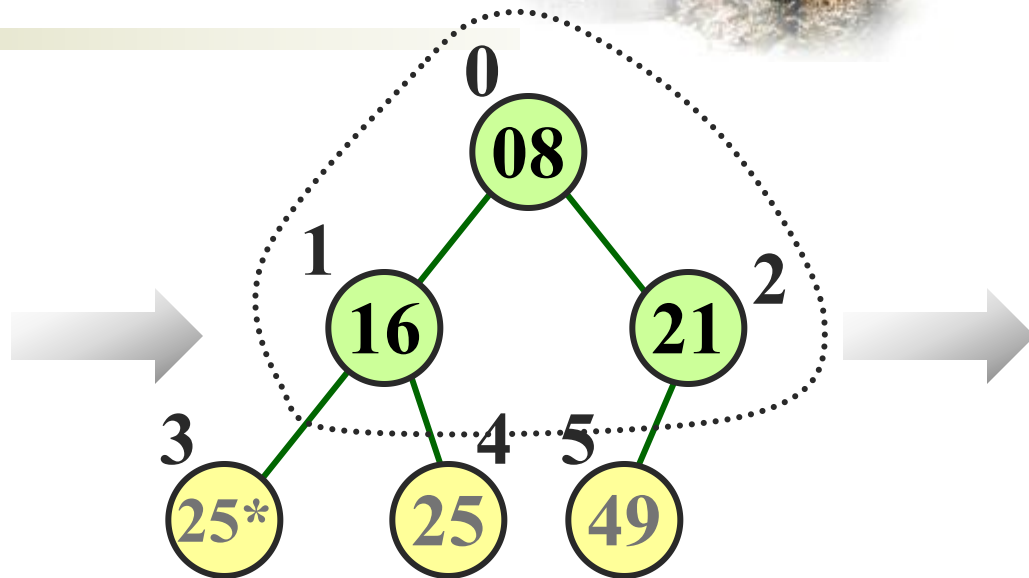
16	25*	21	08	25	49
----	-----	----	----	----	----

交换 0 号与 4 号元素，
4 号元素就位



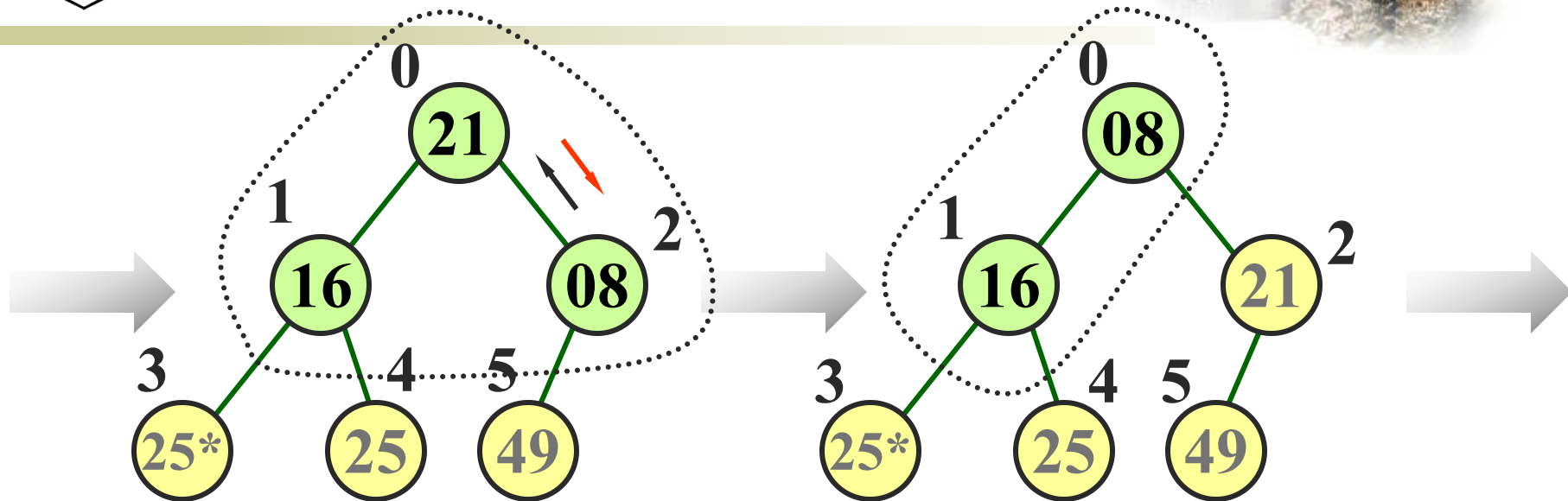
25*	16	21	08	25	49
-----	----	----	----	----	----

从 0 号到 3 号 重新
调整为最大堆



08	16	21	25*	25	49
----	----	----	-----	----	----

交换 0 号与 3 号元素，
3 号元素就位

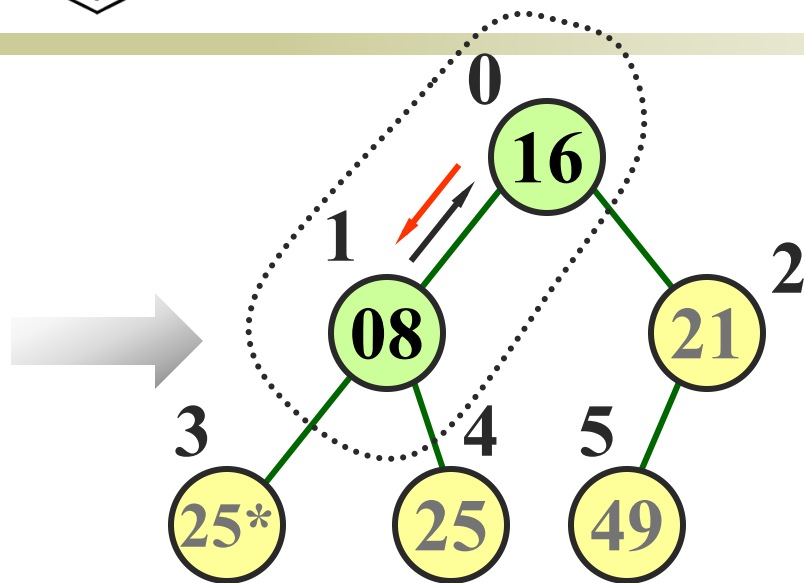


21	16	08	25*	25	49
----	----	----	-----	----	----

从 0 号到 2 号 重新
调整为最大堆

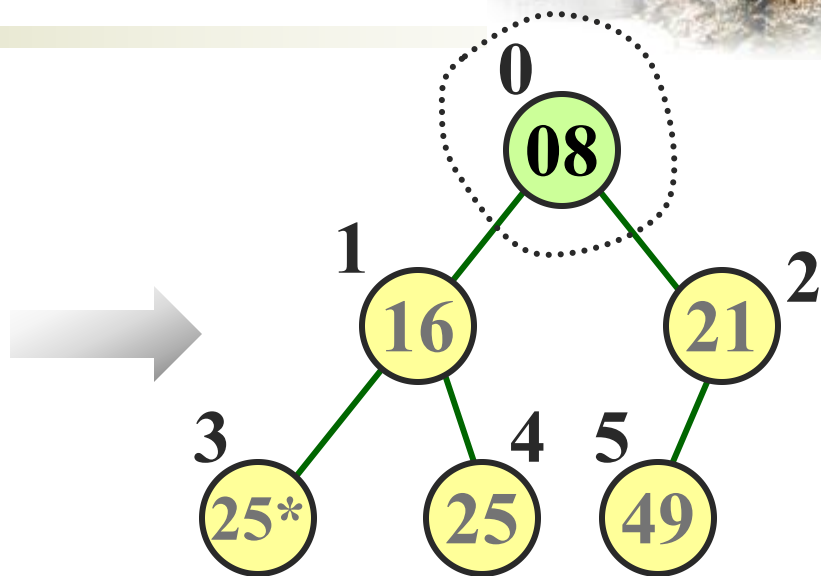
08	16	21	25*	25	49
----	----	----	-----	----	----

交换 0 号与 2 号元素，
2 号元素就位



16	08	21	25*	25	49
----	----	----	-----	----	----

从 0 号到 1 号 重新
调整为最大堆



08	16	21	25*	25	49
----	----	----	-----	----	----

交换 0 号与 1 号元素，
1 号元素就位



堆排序的算法



```
template <class T>
void HeapSort (dataList<T>& L) {
//对表L.Vector[0]到L.Vector[n-1]进行排序, 使得表
//中各个元素按其排序码非递减有序
    int i, n = L.length();
    for (i = (n-2)/2; i >= 0; i--)    //将表转换为堆
        siftDown (L, i, n-1);
    for (i = n-1; i >= 0; i--) {      //对表排序
        L.Swap(0, i); siftDown (L, 0, i-1);
    }
};
```



- 第二个 **for** 循环中调用了 $n-1$ 次 **siftDown()** 算法, 该循环的计算时间为 $O(n\log_2 n)$ 。因此, 堆排序的时间复杂度为 $O(n\log_2 n)$ 。
- 该算法的附加存储主要是在第二个 **for** 循环中用来执行 **元素交换** 时所用的一个临时元素。因此, 该算法的空间复杂度为 $O(1)$ 。
- 堆排序是一个 **不稳定** 的排序方法。



三明治排序



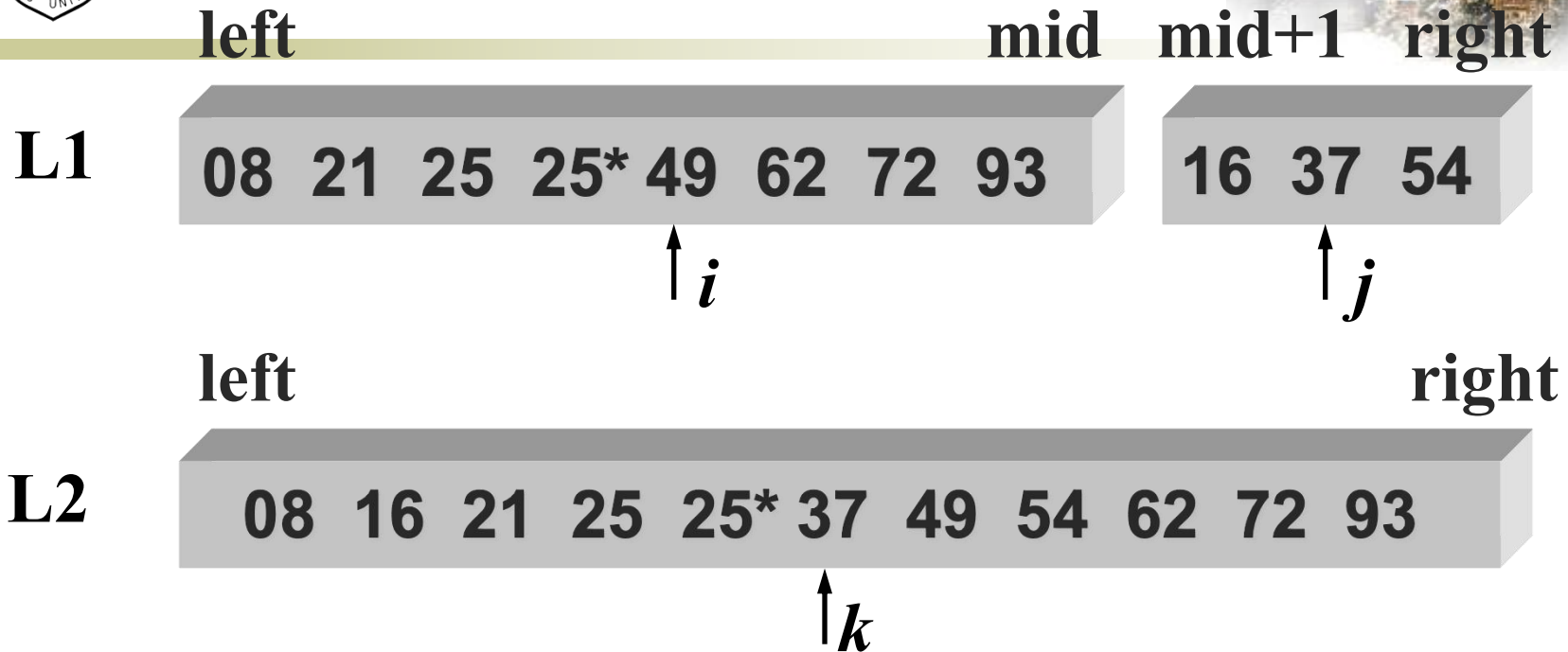
- 设计一个混合排序算法：
 - 当 $n \leq 10$ 时，使用插入排序
 - 当 $n > 10$ 时，使用快速排序
 - 但如果递归深度超过 $\log_2 n$ ，切换到堆排序
- 问题：
 - 这个算法的时间复杂度是多少？
 - 为什么这样设计可能比纯快速排序更好？



9.5 归并排序 (Merge Sort)



- 归并，是将两个或两个以上的有序表合并成一个新的有序表。
- 元素序列L1中有两个有序表Vector[left..mid]和Vector[mid+1..right]。它们可归并成一个有序表，存于另一元素序列L2的Vector[left..right] 中。
- 这种方法称为两路归并 (2-way merging)。
- 变量 i 和 j 分别是表Vector[left..mid]和Vector[mid+1..right]的检测指针。k 是存放指针。



- 当 i 和 j 都在两个表的表长内变化时, 根据对应项的排序码的大小, 依次把排序码小的元素排放到新表 k 所指位置中;
- 当 i 与 j 中有一个已经超出表长时, 将另一个表中的剩余部分照抄到新表中。



迭代的归并排序算法



- 迭代的归并排序算法就是利用两路归并过程进行排序。其基本思想是：

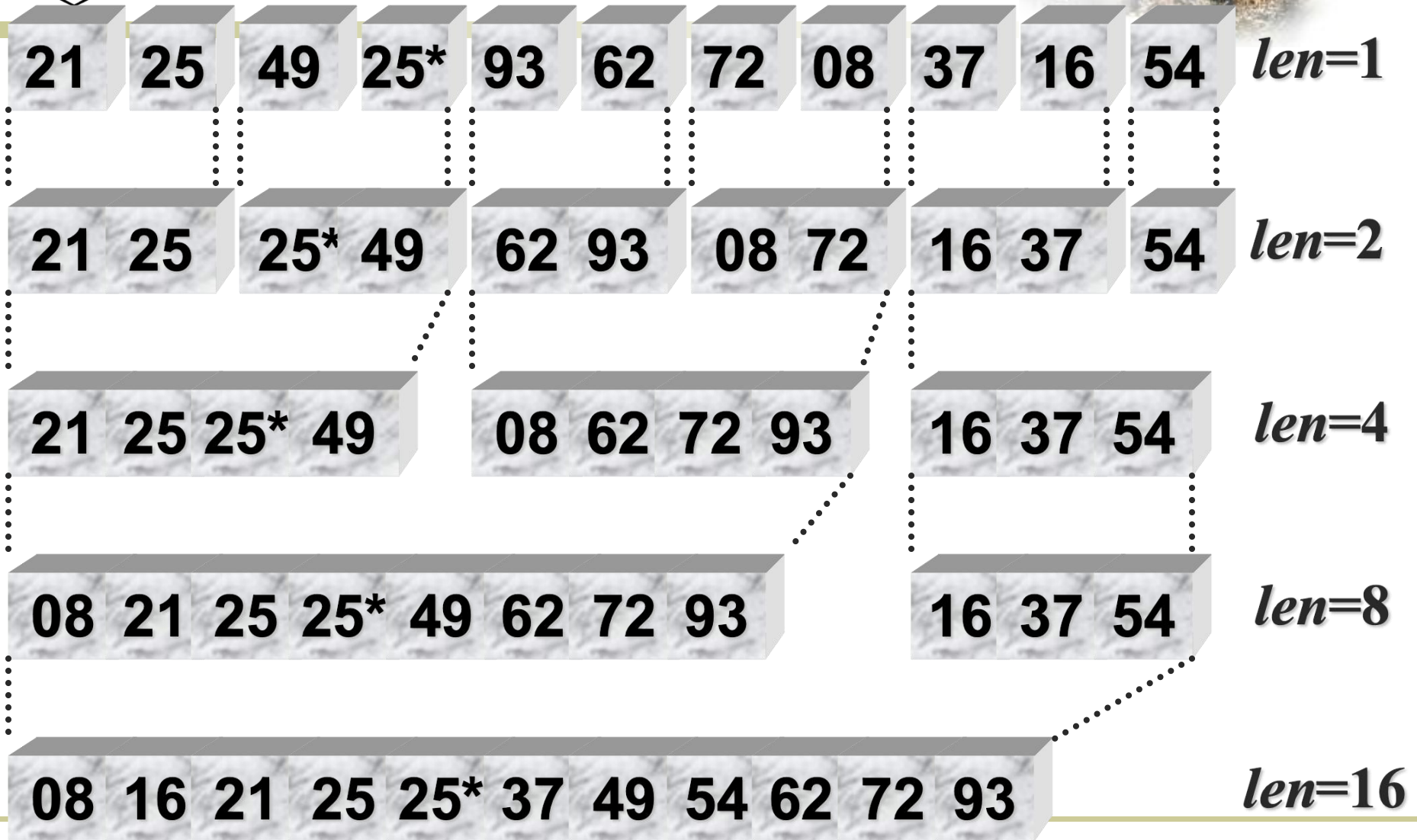
设初始元素序列有 n 个元素，首先把它看成是 n 个长度为 1 的有序子序列（归并项），做两两归并，得到 $\lceil n/2 \rceil$ 个长度为 2 的归并项（最后一个归并项的长度可能为 1）；

再做两两归并，得到 $\lceil n/4 \rceil$ 个长度为 4 的归并项（最后一个归并项长度可以短些）...

如此重复，最后得到一个长度为 n 的有序序列。



迭代的归并排序算法





两路归并算法



```
#include "dataList.h"
template <class T>
void merge (dataList<T>& L1, dataList<T>& L2,
            const int left, const int mid, const int right) {
    //L1.Vector[left..mid]与L1.Vector[mid+1..right]是两个
    //有序表, 将这两个有序表归并成一个有序表
    //L2.Vector[left..right]
    int k, i, j;
    i = left; j = mid+1; k = left;
    //i, j是检测指针, k是存放指针
```



```
while (i <= mid && j <= right)  //两两比较
    if (L1[i] <= L1[j])
        L2[k++] = L1[i++];
    else
        L2[k++] = L1[j++];
while (i <= mid) L2[k++] = L1[i++];
    //若第一个表未检测完, 复制
while (j <= right) L2[k++] = L1[j++];
    //若第二个表未检测完, 复制
};
```



一趟归并排序的算法



- 设 $L1.Vector[0..n-1]$ 中 n 个记录已经分为一些长度为 len 的归并项，将这些归并项两两归并成长度为 $2len$ 的归并项，结果放到 $L2[]$ 中。
- 如果 n 不是 $2len$ 的整数倍，则一趟归并到最后，可能遇到两种情形：
 - ◆ 剩下一个长度为 len 的归并项和另一个长度不足 len 的归并项，可用 **merge** 算法将它们归并成一个长度小于 $2len$ 的归并项。
 - ◆ 只剩下一个归并项，其长度小于或等于 len ，将它直接复制到结果表中。



```
template <class T>
void MergePass (dataList<T>& L1, dataList<T>& L2,
    const int len) {
    int i = 0, j, n = L1.Length();
    while (i+2*len <= n-1) {
        merge (L1, L2, i, i+len-1, i+2*len-1);
        i += 2 * len;           //循环两两归并
    }
    if (i+len <= n-1)
        merge (L1, L2, i, i+len-1, n-1);
    else for (j = i; j <= n-1; j++) L2[j] = L1[j];
};
```



(两路)归并排序的主算法



```
template <class T>
void MergeSort (dataList<T>& L) {
//按元素排序码非递减的顺序对表L中元素排序
    int n = L.Length();
    dataList<T> & tempList (n);           //创建临时表
    int len = 1;
    while (len < n) {
        MergePass (L, tempList, len); len *= 2;
        MergePass (tempList, L, len); len *= 2;
    }
};
```



- 在迭代的归并排序算法中, 函数MergePass()做一趟两路归并排序, 要调用merge()函数 $\lceil n/(2*len) \rceil \approx O(n/len)$ 次, 函数MergeSort()调用MergePass()正好 $\lceil \log_2 n \rceil$ 次, 而每次merge()要执行比较 $O(len)$ 次, 所以算法总的时间复杂度为 $O(n \log_2 n)$ 。
- 归并排序占用附加存储较多, 需要另外一个与原待排序元素数组同样大小的辅助数组。这是这个算法的缺点。
- 归并排序是一个稳定的排序方法。



归并排序

Merge (归并)

把两个或多个有序的子序列合并为一个

2路归并——二合一

k路归并——k合一

归并排序算法

① 若 $low < high$, 则将序列分从中间 $mid = (low + high) / 2$ 分开

② 对左半部分 $[low, mid]$ 递归地进行归并排序

③ 对右半部分 $[mid + 1, high]$ 递归地进行归并排序

④ 将左右两个有序子序列 Merge 为一个

性能

空间复杂度 $O(n)$

时间复杂度 $O(n \log n)$

稳定性 稳定的



排序算法	动物比喻	核心行为	特点
插入排序	蚂蚁搬家	一点一点搬运，逐个放置到位	勤劳但缓慢，适合小规模
快速排序	猎豹捕食	快速突袭，分割包围	快速高效，但可能失败
选择排序	孔雀开屏	每次选出最美的羽毛展示	不断挑选"最佳"元素
归并排序	蜜蜂筑巢	分工合作，完美合并	团队协作，稳定可靠
堆排序	土拨鼠建筑师	构建结构，取出最大，重新调整	结构性很强，保证性能
冒泡排序	青蛙产卵	卵泡缓慢上浮，大的先出	简单直观但效率低
希尔排序	猎豹渐进捕食	从大步到小步，逐步精确	聪明的改进，效率提升



各种排序方法的比较



排序方法	比较次数		移动次数		稳定性	附加存储	
	最好	最差	最好	最差		最好	最差
直接插入排序	$O(n)$	$O(n^2)$	0	$O(n^2)$	√	$O(1)$	
折半插入排序	$O(n \log_2 n)$		0	$O(n^2)$	√	$O(1)$	
起泡排序	$O(n)$	$O(n^2)$	0	$O(n^2)$	√	$O(1)$	
快速排序	$O(n \log_2 n)$	$O(n^2)$	$O(n \log_2 n)$	$O(n^2)$	×	$O(\log_2 n)$	$O(n)$
直接选择排序	$O(n^2)$		0	$O(n)$	×	$O(1)$	
锦标赛排序	$O(n \log_2 n)$		$O(n \log_2 n)$		√	$O(n)$	
堆排序	$O(n \log_2 n)$		$O(n \log_2 n)$		×	$O(1)$	
归并排序	$O(n \log_2 n)$		$O(n \log_2 n)$		√	$O(n)$	



插入排序



选择题

1. 插入排序的最好情况时间复杂度是：
A) $O(n)$
B) $O(n \log n)$
C) $O(n^2)$
D) $O(1)$
2. 在插入排序中，当数据基本有序时，其性能：
A) 最差
B) 最好
C) 与乱序时相同
D) 不确定

填空题

1. 插入排序的核心思想是将每个元素插入到_____中的适当位置。
2. 插入排序的空间复杂度是_____。

问答题

1. 描述插入排序对数组 $[5, 2, 4, 6, 1, 3]$ 的排序过程。
2. 为什么插入排序在小规模数据或基本有序数据中表现良好？



快速排序



选择题

1. 快速排序的平均时间复杂度是：

- A) $O(n)$
- B) $O(n \log n)$
- C) $O(n^2)$
- D) $O(\log n)$

2. 快速排序的最坏情况发生在：

- A) 数组完全有序时
- B) 数组完全随机时
- C) 数组所有元素相等时
- D) 数组元素个数为2的幂时

填空题

1. 快速排序采用_____策略，选取一个基准元素进行分区。
2. 快速排序是不稳定的排序算法，因为交换可能改变_____。

问答题

1. 以数组 $[3, 6, 8, 10, 1, 2, 1]$ 为例，用第一个元素作为基准，描述快速排序的第一趟分区过程。
2. 如何避免快速排序的最坏情况？给出两种方法。



选择排序



选择题

1. 选择排序的时间复杂度是：

- A) $O(n)$
- B) $O(n \log n)$
- C) $O(n^2)$
- D) $O(1)$

2. 选择排序的主要操作是：

- A) 交换相邻元素
- B) 比较和交换
- C) 递归分治
- D) 建立堆结构

填空题

1. 选择排序每趟从待排序序列中选择_____元素放到已排序序列末尾。
2. 选择排序的比较次数与初始序列的顺序_____。

问答题

1. 对数组 [64, 25, 12, 22, 11] 进行选择排序，写出每趟排序后的数组状态。
2. 选择排序和冒泡排序的主要区别是什么？



归并排序



选择题

1. 归并排序的时间复杂度是：

- A) $O(n)$
- B) $O(n \log n)$
- C) $O(n^2)$
- D) $O(\log n)$

2. 归并排序的空间复杂度是：

- A) $O(1)$
- B) $O(\log n)$
- C) $O(n)$
- D) $O(n^2)$

填空题

1. 归并排序采用_____策略，将数组分成两半分别排序后再合并。

2. 归并排序是_____排序算法（稳定/不稳定）。

问答题

1. 描述归并排序对数组 [38, 27, 43, 3, 9, 82, 10] 的完整排序过程。

2. 归并排序为什么需要额外的 $O(n)$ 空间？能否实现原地归并？



第9章书面作业



提交时间：12月23日，上课前提交

第9章：

- (1) 第440页，9.2：第(1) – (8)